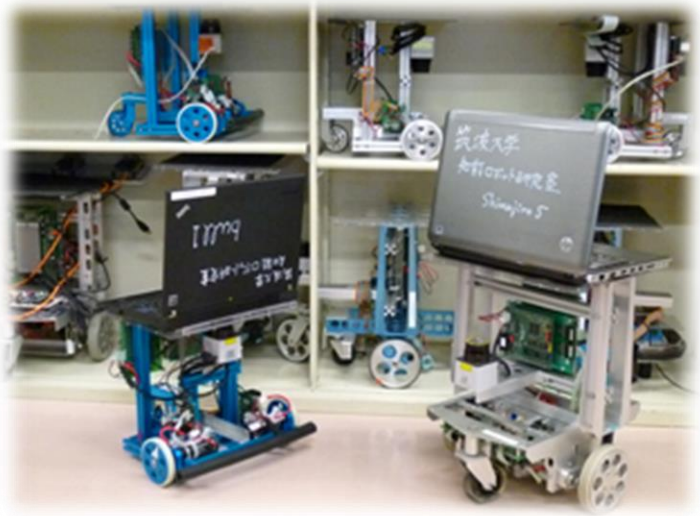
ロボットの扱い方
(+Dockerの導入)



担当 MYJ, TCB, KYM, ITK

筑波大学 知能ロボット研究室
University of Tsukuba, Intelligent Robot Laboratory

アウトライン

■ 「山彦」の概要

- システム構成

■ 「山彦」の準備・電源の入れ方

- ロボット・バッテリ置き場@SB1028
- 電源の入れ方
- 充電方法

■ 「Docker」を使う

- Docker概要
- 使用方法

■ 「山彦」を動かしてみる

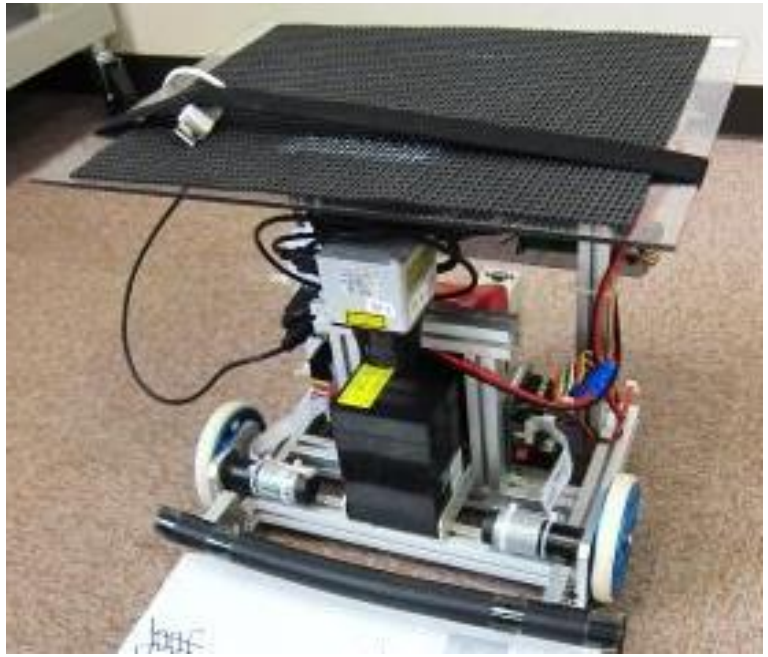
- コントローラで動かす
- コマンドラインで動かす

2

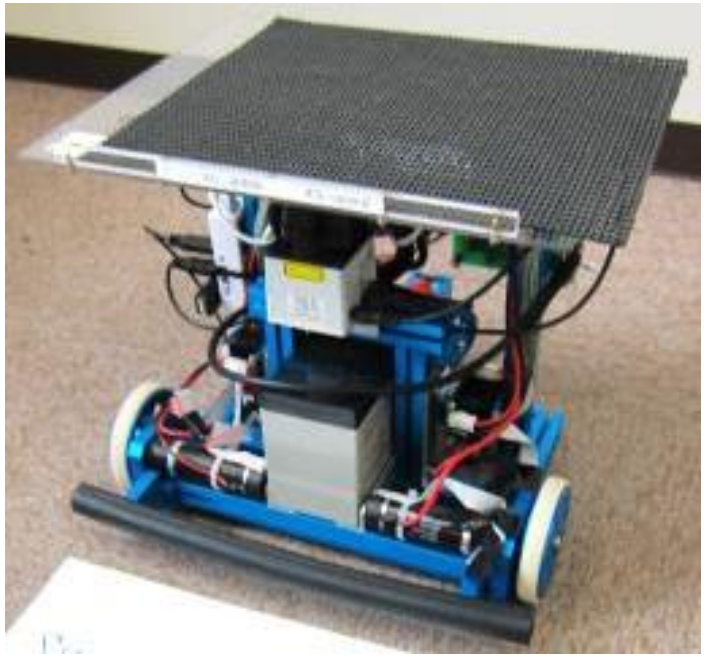
「山彦」の概要

3 実験に使用するロボット「山彦」

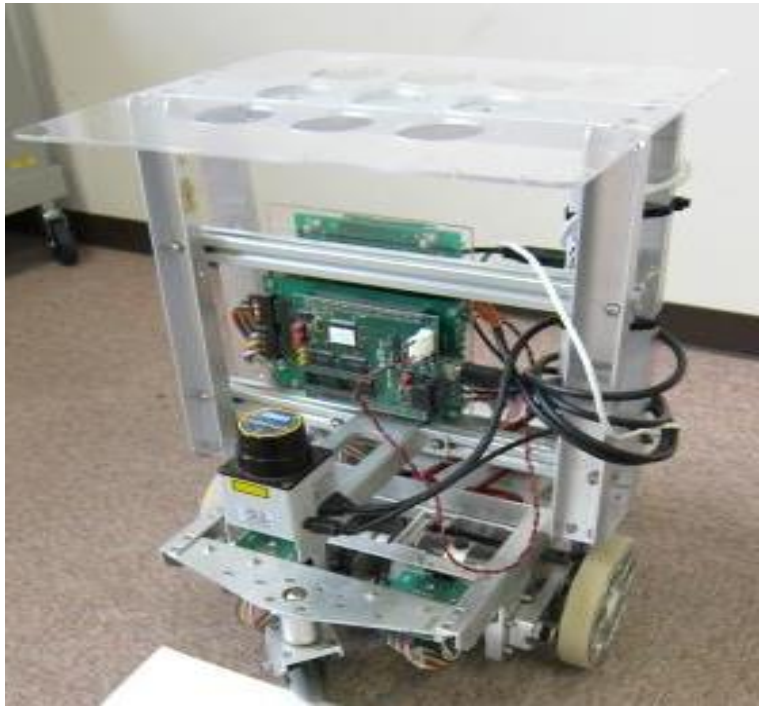
- 3タイプ
- 大きさ・最高速度は違うが基本構成は同じ



Beego
ビーゴ

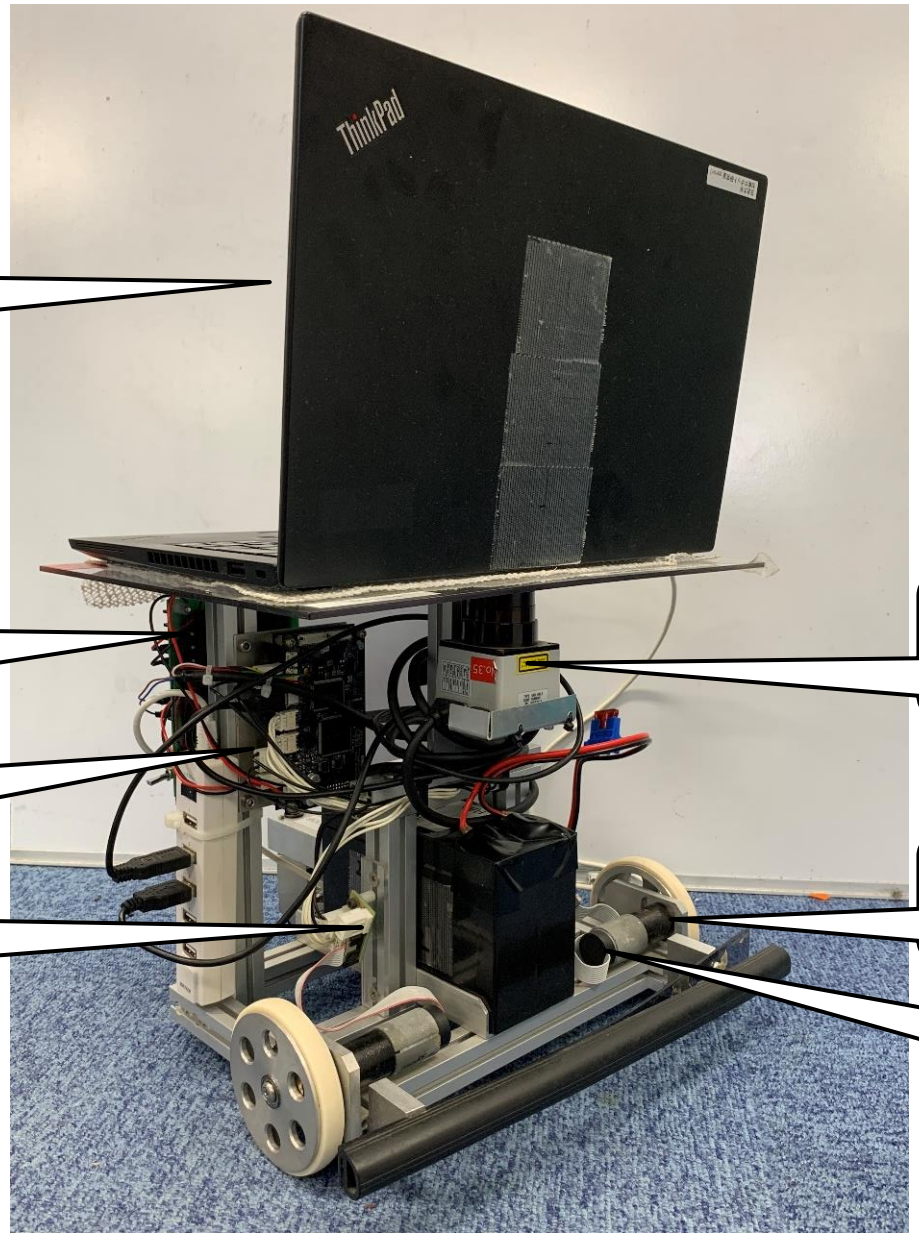


Speego
スピーゴ



M1
エムワン

4 山彦のハードウェア構成



ノートPC
メイン処理

電源基板
電圧変換⇒センサ・コントローラ

モータコントローラ
車輪の回転制御

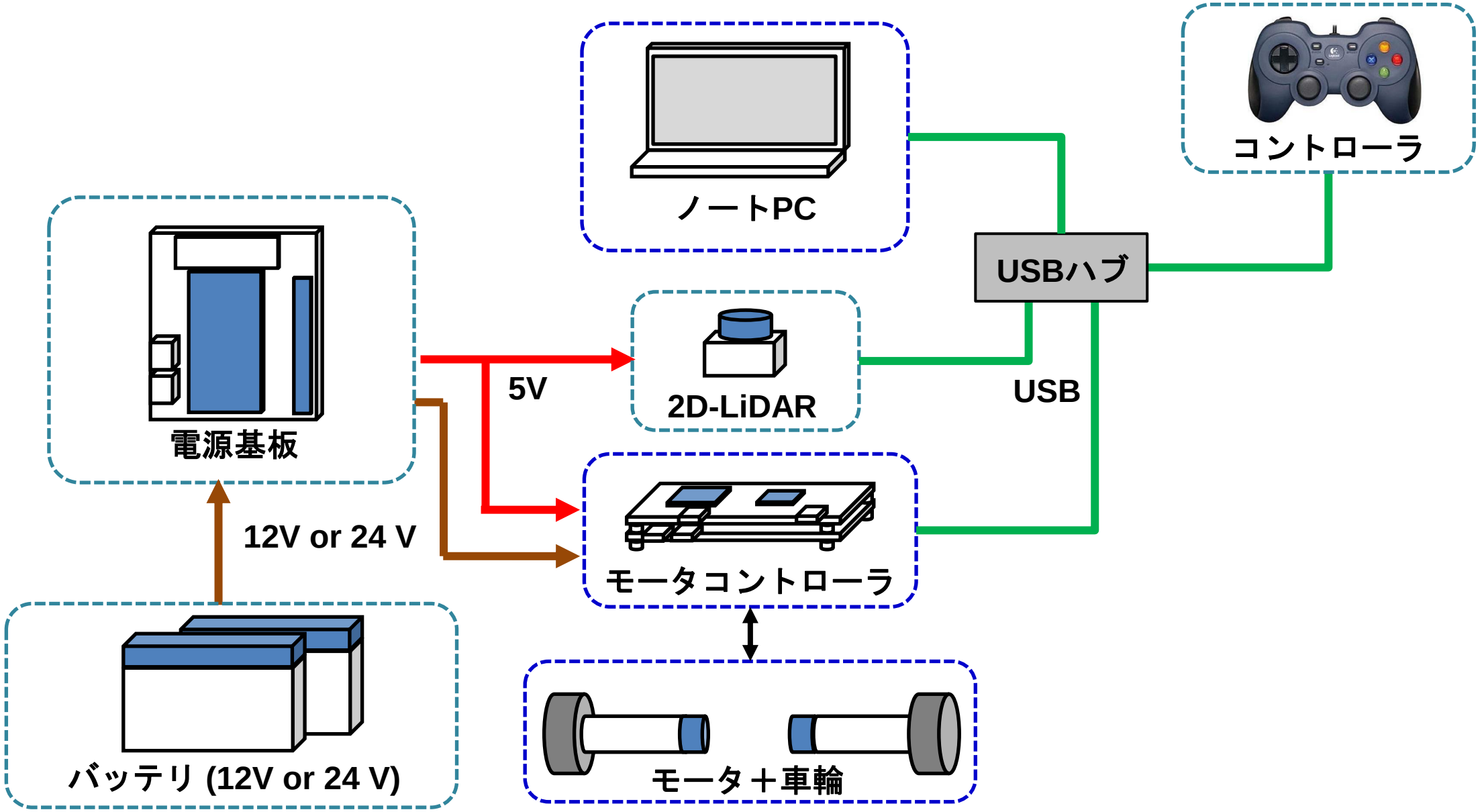
バッテリ
電源供給

2D-LiDAR
周囲までの2次元距離情報計測

モータ＋車輪
回転・移動

エンコーダ
モータの回転数計測

5 山彦のハードウェア構成



6

「山彦」の準備・電源の入れ方

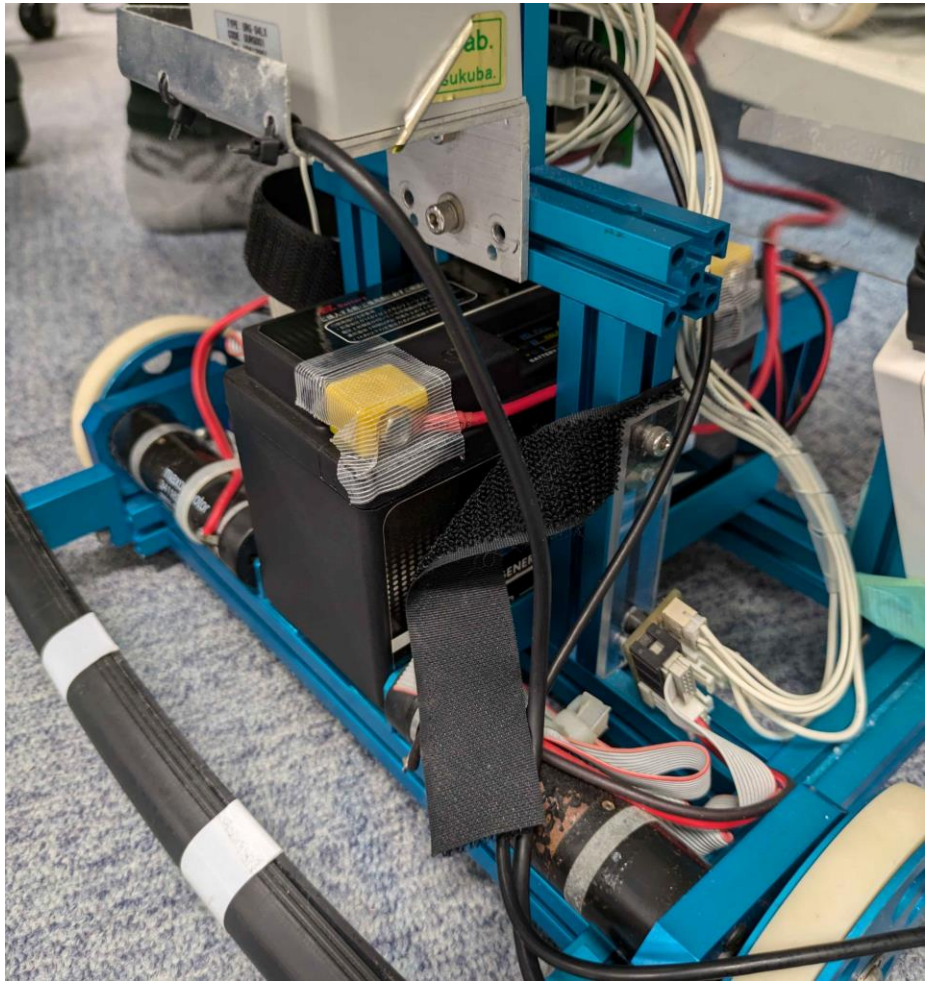
7 ロボットとバッテリ置き場@SB1028

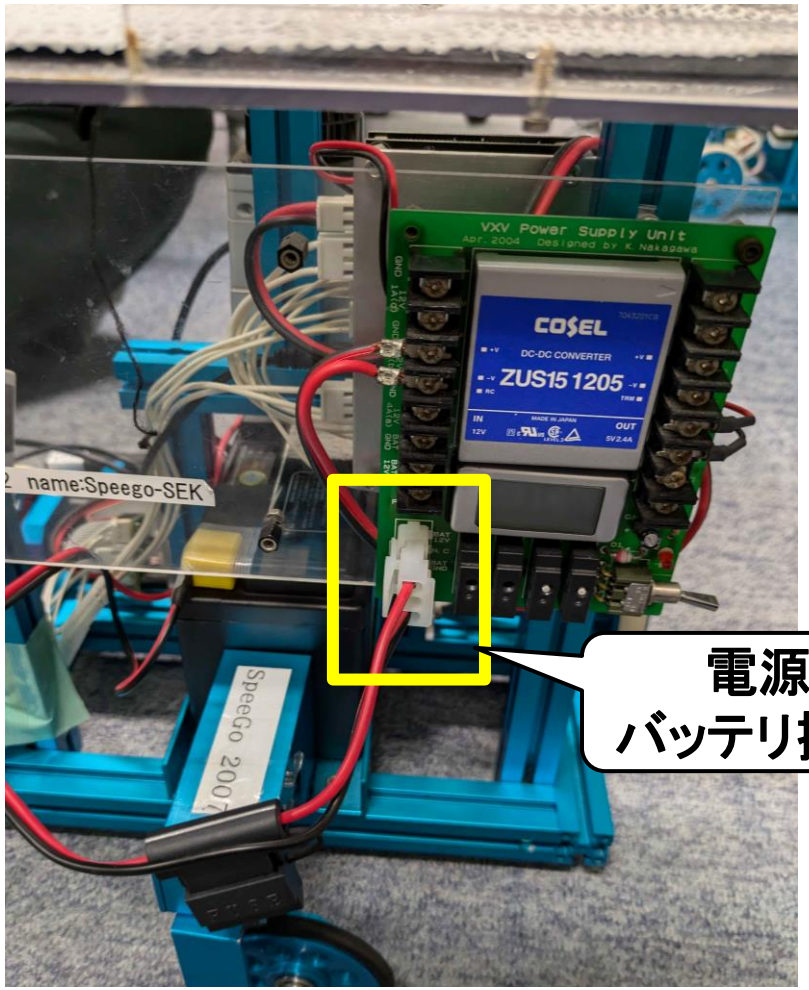
- セミナー開始前にSB1028室から自分のロボットとバッテリを持ってくる
  - セミナー後の返却も同様

ロボット：SB1028入って右側の棚

バッテリ：SB1028入って左側の棚

8 バッテリの取り付け・電源基板への接続（Beego・Speego）

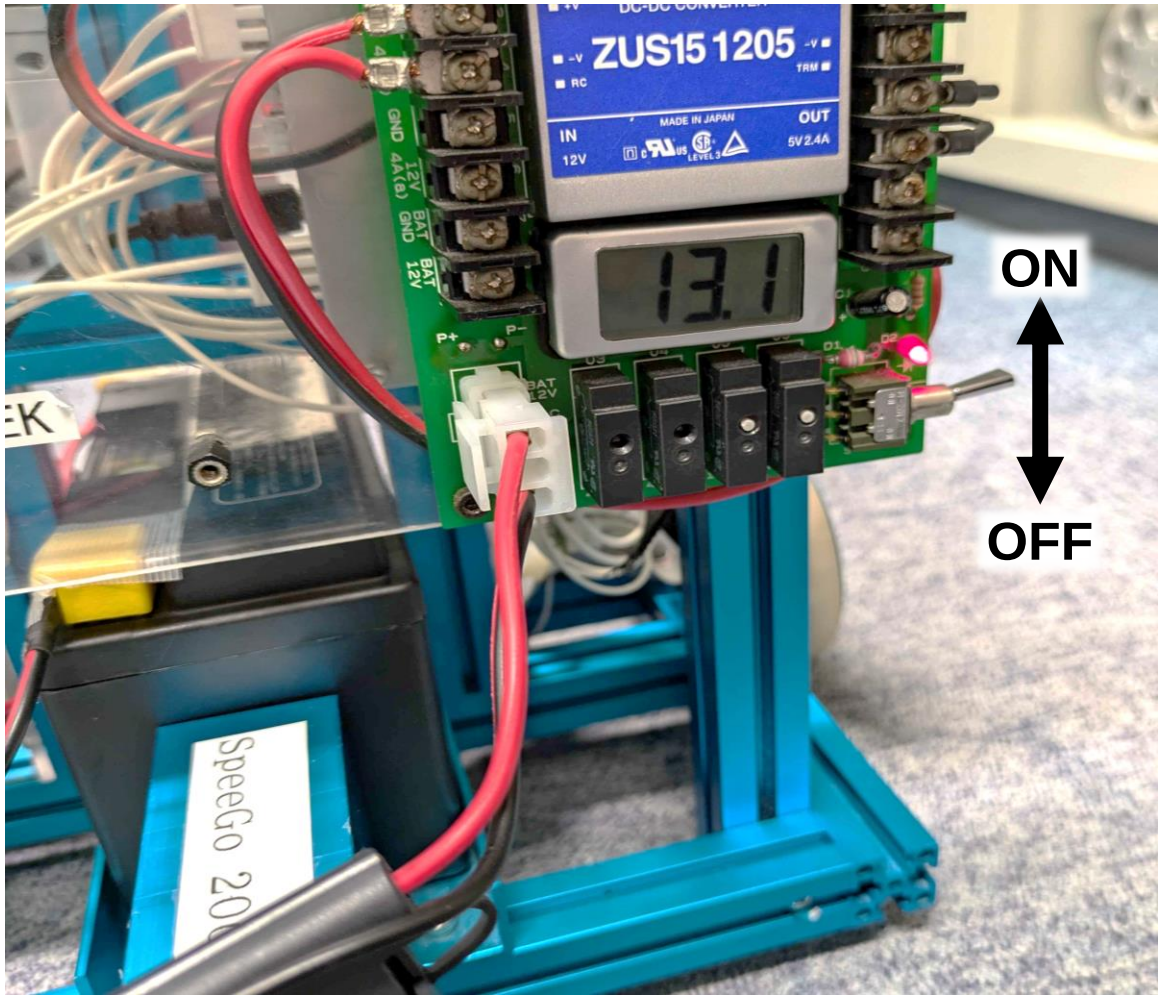




電源基板
バッテリ接続端子

9 電源の入れ方（Beego・Speego）

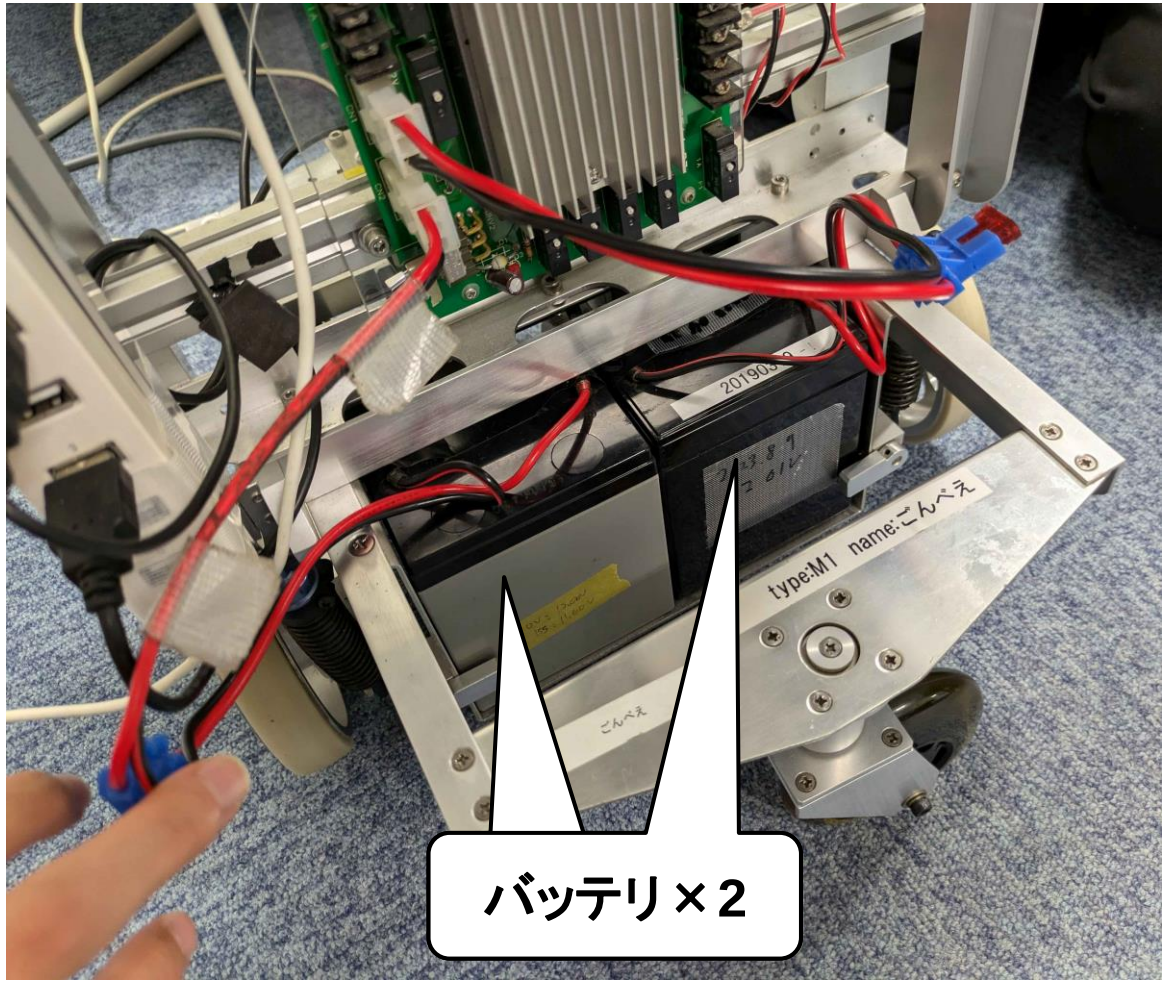
□ 電源基板のスイッチをON

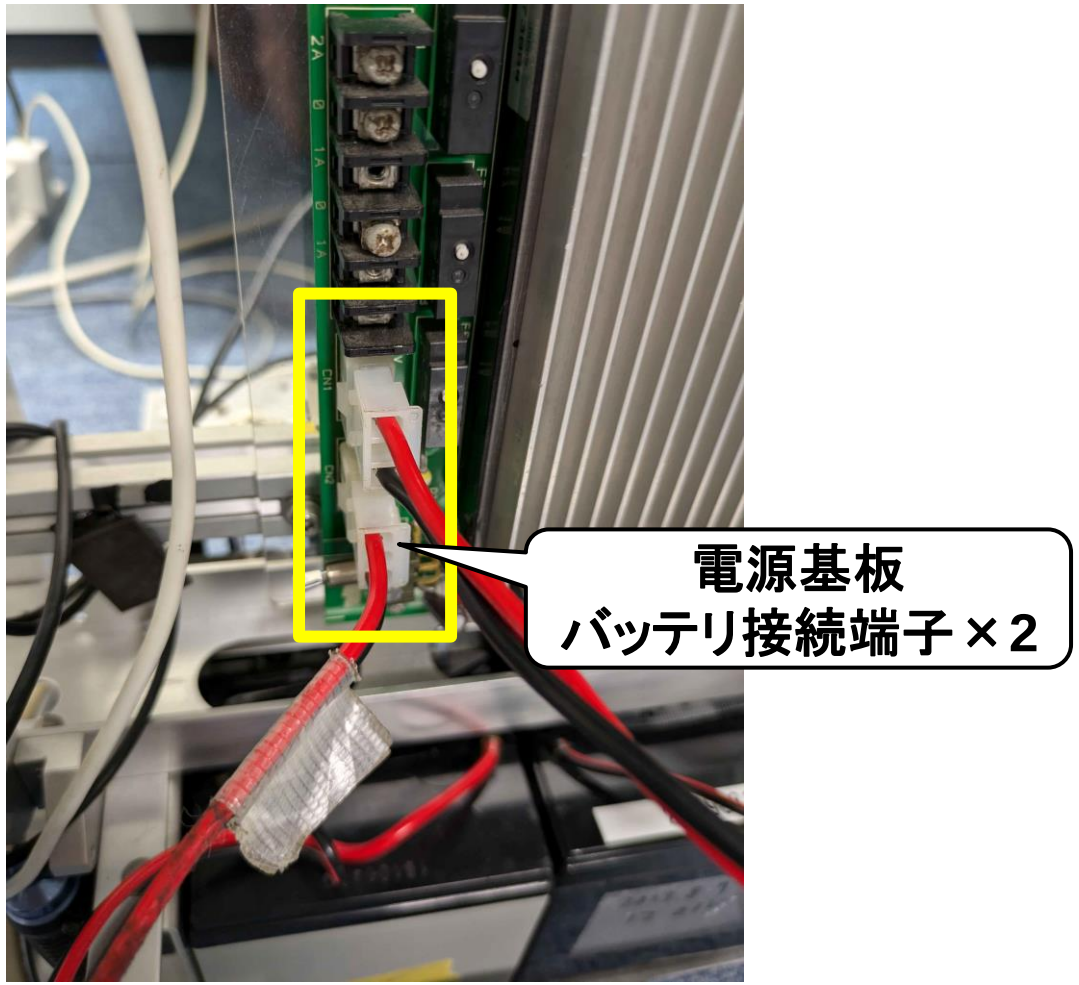


□ 電圧表示が12V以下の場合は充電

- 本体のバッテリランプでも
  残量を3段階で確認できます

10 バッテリの取り付け・電源基板への接続（M1）

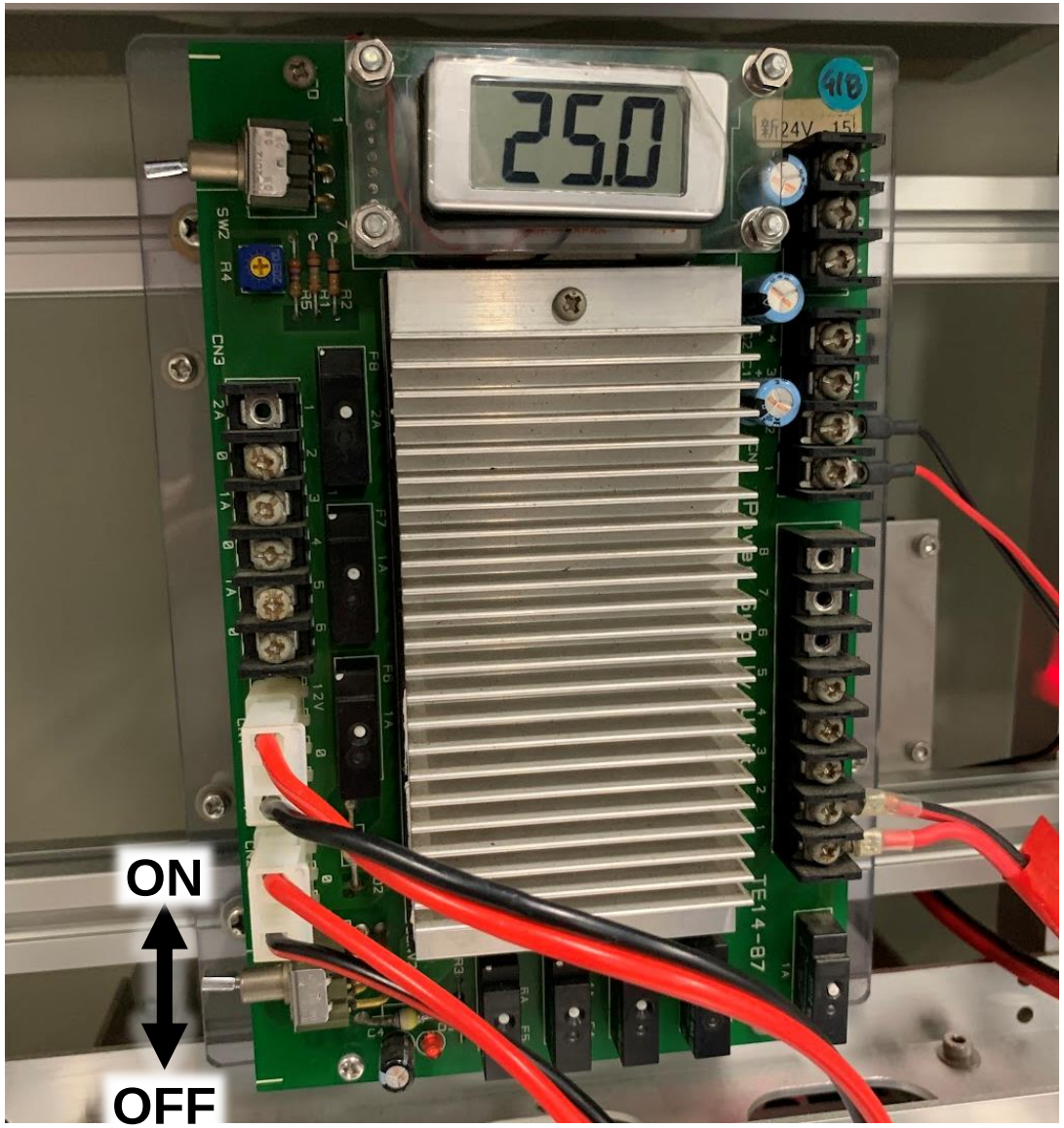




11 電源の入れ方（M1）

- 電源基板のスイッチをON

- 電圧表示が24V以下の場合は充電



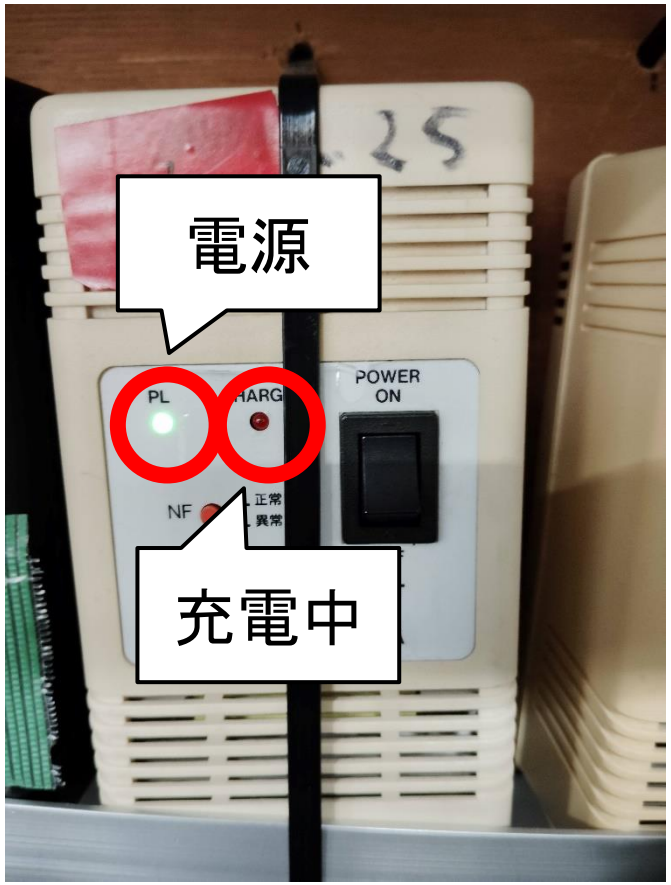
充電方法（ Beego･Speego ）

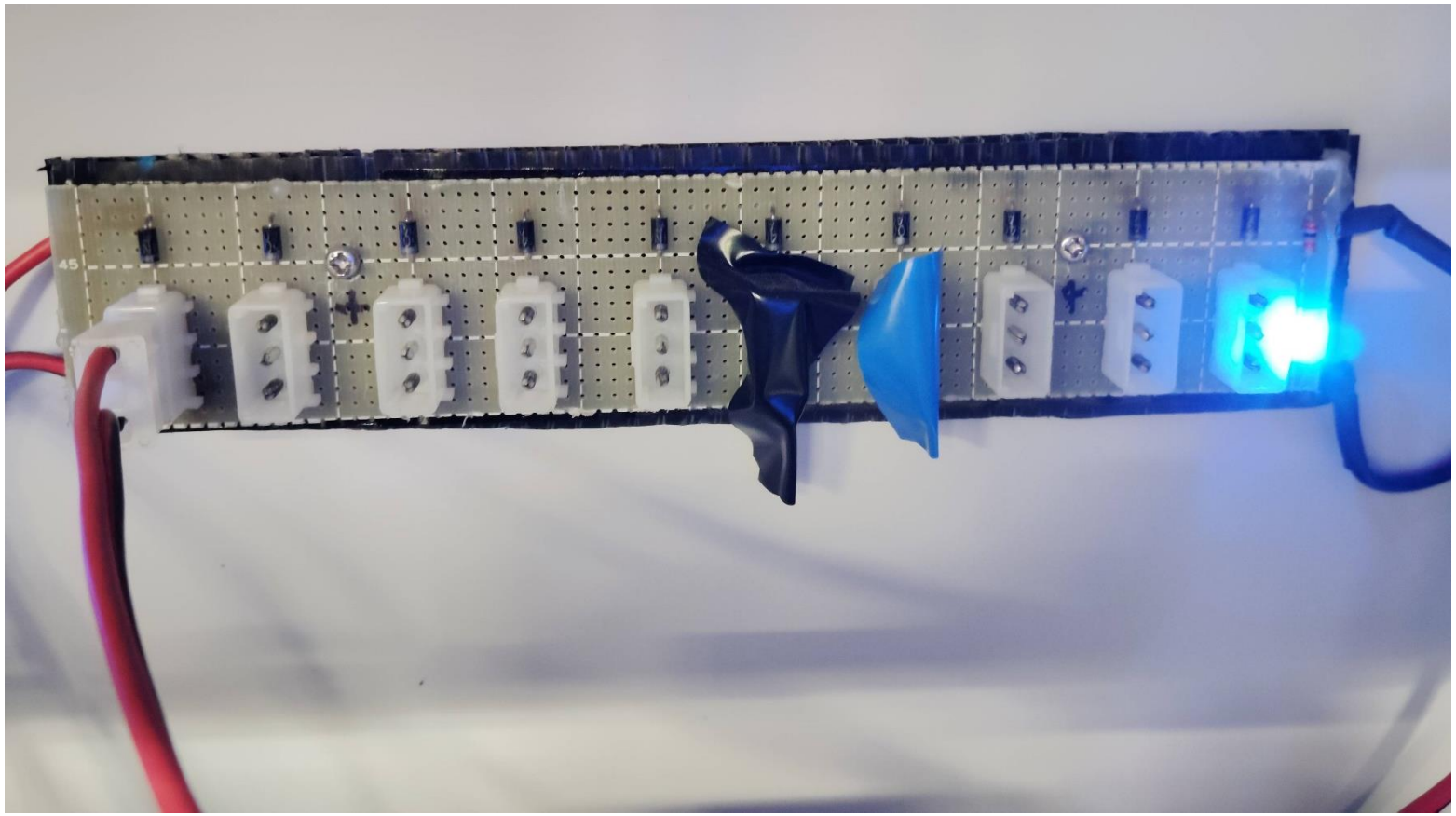
□ケーブルを写真の充電器に刺す

13 充電方法（M1）

- ケーブルを写真(左)の充電器に刺す

- 充電が終わって充電中ランプが消灯したら，写真(右)に刺し替える





14

「Docker」を使う

Docker概要

Dockerとは

- アプリケーションを仮想環境で動かすためのもの
- ターミナル上で仮想環境を立ち上げると，仮想環境内でコマンドを実行できる

Dockerの用語

- コンテナ : アプリケーションとその依存関係を含む軽量な仮想環境
- イメージ : コンテナを作成するためのテンプレート
- Dockerfile : イメージを作成するためのコマンド等を書いたファイル
- Docker Compose : 複数のコンテナを一括管理するためのツール

今回Dockerを使用する理由

- Ubuntu22.04でROS1を使用するため(ROS1は本来ubuntu20.04でしか実行できない)

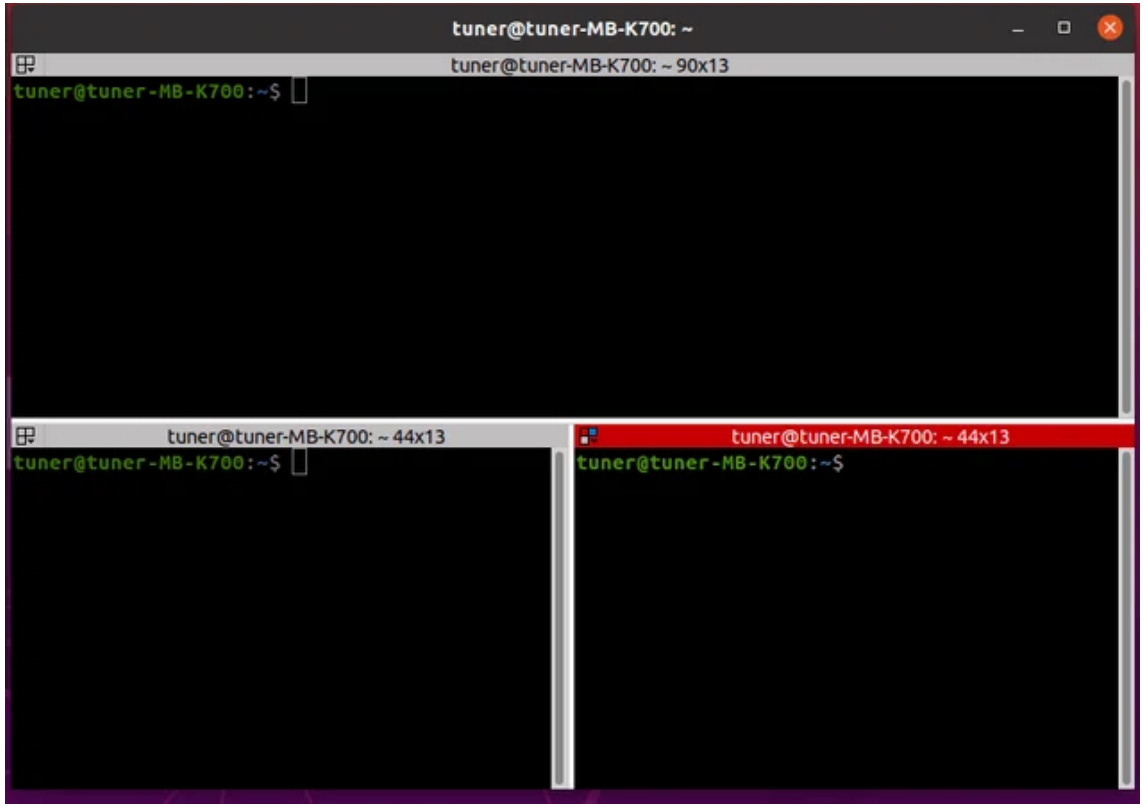
ノートPCでターミナルを開く

複数のプログラムを実行

- 複数のターミナルを利用します

Terminatorが便利（普通のターミナルでももちろん構いません）

- 起動:　　Ctrl + Alt + t
- 縦分割:　Ctrl + Shift + e
- 横分割:　Ctrl + Shift + o
- 小窓削除: Ctrl + Shift + w
- 小窓移動: Ctrl + Tab



17 Dockerを使う

Dockerのコンテナの立ち上げ

1. $ cd $HOME/yamasemi_ws ：山セミのワークスペースへ移動
2. $ sudo –E docker compose run –rm ros ：コンテナの起動
3. /# [何かしらのコマンド1] 👈 /# で始まるコマンドはコンテナ内で実行している
4. /# [何かしらのコマンド2]
5. /# exit : ターミナルから退出

- 今後 /# で始まるコマンドがあれば，上記の手順でコンテナを起動してから実行する

山彦を動かすための準備

- ワークスペース内のパッROSケージのビルド
- パッケージを追加したりCmakeLists.txtを変更したときに行う

1. /# cd /root/yamasemi_ws
2. /# catkin_make

Docker Imageを更新する

Docker Imageを更新するタイミング

- パッケージの依存関係を変更したとき
- Docker fileを変更したとき

更新方法

1. `$ cd $HOME/yamasemi_ws` ：山セミのワークスペースへ移動
2. `$ sudo –E docker compose build ros` ：イメージのビルド

「山彦」を動かしてみる

- 動作確認
  - とりあえず動かしてみましょう
  - コマンドの中身は後で理解するので構いません

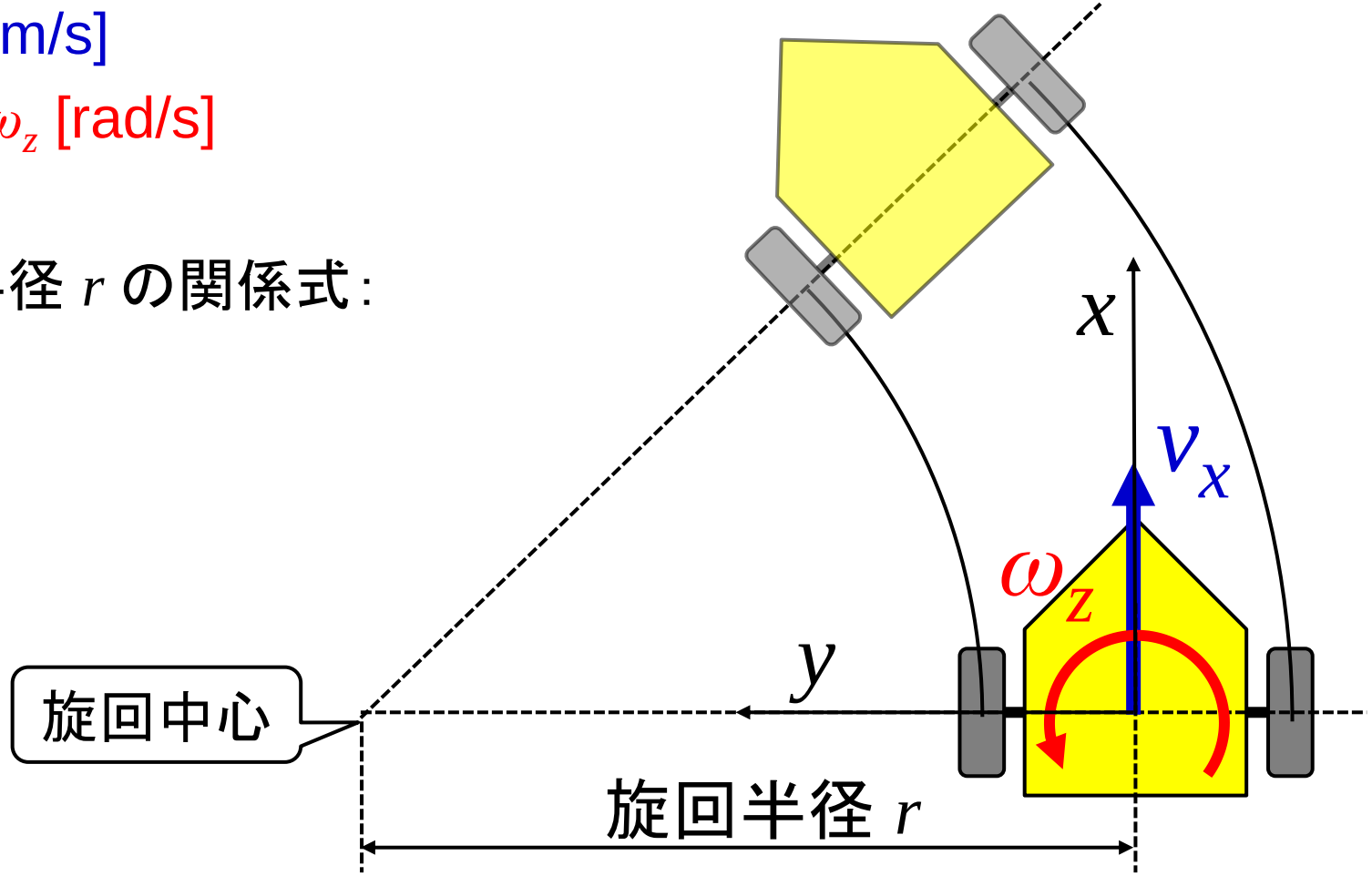
移動体の運動学（独立二輪駆動）

- 2つの車輪の回転によって動きが決まる

- 一般的に以下の**速度指令値**を与えてロボットを走行させる
  - 並進速度 v_x [m/s]
  - 回転角速度 ω_z [rad/s]

  - v_x, ω_z, 旋回半径 r の関係式：

$$v_x = r\omega_z$$



1. コントローラで動かす

ターミナル① 👈必ず使うコマンド

1. /# roscore

ターミナル② 👈必ず使うコマンド

1. /# source /root/yamasemi_ws/devel/setup.bash
2. /# roslaunch ypspur_ros “robot-name”.launch
   - 例）Beegoの場合
     /# roslaunch ypspur_ros beego.launch

ターミナル③

1. /# source /root/yamasemi_ws/devel/setup.bash
2. /# roslaunch yamasemi_ex joypad.launch

終了時(すべてのターミナル)👈必ず使うコマンド

- Ctrl + c

コントローラ操作

2. ターミナルでコマンドラインで動かす

□ ターミナル① 👈 必ず使うコマンド

1. /# roscore

□ ターミナル② 👈 必ず使うコマンド

1. /# source /root/yamasemi_ws/devel/setup.bash
2. /# roslaunch ypspur_ros “robot-name”.launch
   - 例）Beegoの場合
     /# roslaunch ypspur_ros beego.launch

□ ターミナル③

- 例）v_x=0.1, ω_z=0.2 の速度指令値を10Hzで送信する
- /# rostopic pub /ypspur_ros/cmd_vel geometry_msgs/Twist -r 10 -- '[0.1, 0.0, 0.0]' '[0.0, 0.0, 0.2]'
  - Ctrl + cで停止させる

並進速度 $[v_x, v_y, v_z]$ =[0.1, 0.0, 0.0]

10 Hz

回転角速度 $[\omega_x, \omega_y, \omega_z]$ =[0.0, 0.0, 0.2]

※最大 v_x=0.5, ω_z=0.5 くらいまで

※全部手で打つと間違うこともあります
途中まで打って，tab補間が便利です

課題

- v_x=0.2で旋回半径1.0 mで走らせる